

Bayesian Classification

① Probability & conditional probability

- Probability $P(A)$ = Likelihood of event A
- Conditional prob $P(A|B)$ = prob. of A given B has happened
- Chain Rule $P(A|B) = P(A|B)P(B)$
 $P(A, B) = P(B|A)P(A)$

Ex: Bag contains 4 red, 5 blue & 3 green balls. Prob of blue ball if drawn at random.

$$\text{Total balls} = 4 + 5 + 3 = 12$$

$$P(A) = \frac{5}{12} = \frac{\text{No of favourable outcomes}}{\text{Total outcomes}}$$

Ex: ~~box contains 3 white & 2 black balls~~ Refer sheet

- Joint probability → if not given and A & B are independent
 $P(A \cap B) = P(A) \cdot P(B)$

Ex: $P(A) = 0.4$, $P(B) = 0.5$ $P(A \cap B) = 0.4 \times 0.5$

- If A and B are dependent in joint probability we can use above given chain rule

$$P(A \cap B) = P(A) \times P(B|A) \text{ OR } P(B) \times P(A|B)$$

- Find $P(A \cup B)$ union given $P(A)$, $P(B)$ and $P(A \cap B)$

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

- Test sensitivity → detect true positives, for example given test sensitivity = 0.99 = 99% test correctly classifies individuals who actually have the disease.

$$\text{Sensitivity} = P(\text{Positive} | \text{Disease}) = 0.99$$

Metric	Focus	Formula
Sensitivity	True positive rate (TPR) (detects)	$TP / (TP + FN)$
Specificity	True negative rate (TNR) (excludes)	$TN / (TN + FP)$

② Bayes' Theorem

- Core concept = Formula = $P(A|B) = \frac{P(B|A) \times P(A)}{P(B)}$

Posterior → update probability after evidence

Labels: $P(B|A)$ = Likelihood given A is true; $P(A)$ = prior; $P(B)$ = Joint probability total prob.; $P(A|B)$ = observing evidence

Prior - Initial Belief before evidence
 Likelihood - Probab. of evidence given hypothesis
 Evidence $\rightarrow$ Total prob. of evidence

$\rightarrow$ what is $P(B)$ is not given directly, it needs to be computed first.
 Ex: example we need to find out
 $P(\text{Positive})$ for disease

& we are given $P(D) = \text{prob. of disea} = 0.01$

Sensitivity = $P(T|D) = 0.99$

False true rate = $P(T|\neg D) = 0.05$

Prob. of $P(\neg D) = 1 - P(D) = 0.99$

~~$P(T) = P(\text{true positive})$~~

$$P(\text{positive}) = \underbrace{P(T|D)P(D)}_{\text{True positive}} + \underbrace{P(T|\neg D)P(\neg D)}_{\text{False positives}}$$

$$= (0.99 \times 0.01) + (0.05 \times 0.99) = 0.0594$$

Sometimes, specificity is given = True negative rate

To compute false positive rate = $1 - \text{specificity}$

$\rightarrow$ Finding Accuracy in terms of threshold in Bayesian classification
 model.

$$\text{Accuracy} = \frac{TP + TN}{\text{Total}}$$

TP = True positives
 TN = True negatives

Naive Bayes classification

$\rightarrow$ Likelihood $P(x|x)$ compute with multiple features - as assumption of independence holds $\rightarrow$ conditional independence and multiply individual feature probabilities.

① To compute for some classification

$\rightarrow$ Convert data into frequency table

$\rightarrow$ Create likelihood table by finding the probabilities.

Compute Probability of Error

Bayes Error rate to Bayesian classifier

Say class 1 (w_1) $\rightarrow N(\mu_1=0, \sigma_1^2=1)$

Say class 2 (w_2) $= N(\mu_2=2, \sigma_2^2=1)$

Equal priors $= P(w_1) = P(w_2) = 0.5$

$\rightarrow$ Decision Boundary $x=B \rightarrow$ posterior prob or equal

$$P(x|w_1) \times P(w_1) = P(x|w_2) \times P(w_2)$$

$$\frac{1}{\sigma_1 \sqrt{2\pi}} e^{-\frac{x^2}{2}} = \frac{1}{\sigma_2 \sqrt{2\pi}} e^{-\frac{(x-2)^2}{2}}$$

Solution = 7)

Compute misclassification Prob

Error for $w_1 \rightarrow x > 1$

$$P(x > 1 | w_1) = 1 - \Phi(1) = 1 - \frac{0.8413}{\sqrt{2\pi}} = 0.1587$$

$$\frac{1}{\sqrt{2\pi}} \times 0.606 = 0.2764$$

⇒ How many model parameters to specify likelihood?

Suppose image pixel 30×30 $n = 900$

$$P(x_1, \dots, x_n | Y) = 2(2^n - 1) = 2(2^{900} - 1)$$

$$P(x_1 | Y), \dots, P(x_n | Y) = 2n = 2 \times 900$$

Correlation coeff $r = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y} \rightarrow$

→ multivariate density → Gaussian lie in cloud centered on the mean
 μ → Parameter estimation center of ellipse

→ accounts for complex dependencies → covariance & correlation b/w features

→ Compute conditional risks for each action

$$R(a_i | x) = \sum_{j=1}^2 \lambda(a_i | w_j) \cdot P(w_j | x)$$

Neural Networks

CNN

Convolution Layer → output spatial size formula = $\left\lceil \frac{N - F + 2P}{S} \right\rceil + 1$

⇒ depth of output is the no. of filters used.
 $N = \text{ht or width}$
 $F = \text{filter (kernel size)}$
 $P = \text{padding}$
 $S = \text{stride}$
 * if avg. pooling gradient flows $1/4$ ex. for each distributed.

⇒ Max pooling only passes the gradient to the position that had the maximum value → all other places its zero.

⇒ When backpropagating thru convolution layer → gradient at each ~~state~~ position depends on the kernel value that multiplied it during the forward pass.

Max pooling → gradient to max value, Avg → equally to all positions

Pooling = down sampling ops on feature map
Gradient at underlined position = Gradient from o/p x corresponding kernel

Input $\rightarrow$ convolution — Activation (ReLU) $\rightarrow$ max pooling

ReLU helps $\Rightarrow$ ① Adds non-linearity ② Solves vanishing gradient problem
③ create sparse representations ④ often leads to faster convergence.

ReLU $\rightarrow$ learn more complex features followed by pooling
reduces spatial size — network more efficient & robust to small translations.

for max pooling & ReLU can be swapped without affecting o/p
 ℓ_2 both are monotonic & element-wise.

Downsample + preserve spatial invariance = Avg & max
For max: 5×5 shared wts + bias = 26 param
And we 20 features $= 20 \times 26$

$H_{max} = \max_{i,j} C_{ij}$
greatly reduces # parameters
Shared wts use same wts & Bias for each of hidden neurons
All neurons in the first hidden layer detect exactly SAME FEATURE
just at different locations of the i/p image. — said to define
KERNEL OR FILTER

Aim of conv. layer — learn the filter or kernel matrix

Why use PADDING $\rightarrow$ Image shrinks, pixels in corner used only once, data in the corners thrown away.
 $\rightarrow$ Padding + I/p image before convolution
 $\rightarrow$ Append zeros around the Boundary of the i/p

How much to pad?

$$\text{Padding} = p = \frac{f-1}{2}$$

if kernel = 3×3 5×5
 $p = \frac{3-1}{2} = 1$ $p = \frac{5-1}{2} = 2$

Valid Padding $\rightarrow$ not applied i/p size $\neq$ o/p size $p=0$

Same padding $\rightarrow$ applied = i/p size = o/p size $p = \frac{f-1}{2}$

Stride skip by how many pixels

Pooling — subsampling, speed up computation, applied on each channels

PCA

⇒ Eigenvalues & eigenvectors — basis vectors just rotated by matrix
scale them

$$\begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix}$$

⇒ Eigenvectors are not unique (unit vectors are used)

⇒ $A = \Phi \Lambda \Phi^T$ Λ = diagonal matrix
eigen decomposition — Rotation matrix

Example

$$A \vec{x}_i = \lambda_i \vec{x}_i \approx A(\alpha \vec{x}_i) = \lambda_i (\alpha \vec{x}_i)$$

⇒ matrix = n dims = n diff unit eigenvectors

⇒ How covariance & PC components related?

PC algo finds covariance matrix — 2 maximum variance components are taken!

Covariance matrix

Covariance matrix $\Sigma v = \lambda v$ eigenvalue
eigenvector

$$\begin{bmatrix} \text{Var}(X) & \text{Cov}(X,Y) \\ \text{Cov}(X,Y) & \text{Var}(Y) \end{bmatrix}$$

Correlation matrix $\begin{bmatrix} 1 & \text{corr}(X,Y) \\ \text{corr}(X,Y) & 1 \end{bmatrix}$

⇒ Projection method, good, sometimes like PCA

⇒ If you have n samples & d features and center the data data matrix can have at most n-1 → linearly independent rows — centering reduces the rank by 1

⇒ Find PCs of a training set — use SVD

Full SVD & Economy SVD

Hard Voting Classifier

Random Forest

⇒ Combines preds — multiple — individual classifiers — Majority vote. — uses predicted class labels

⇒ Soft voting → avg. predicted probabilities and pick the class with the highest avg. probability.

→ slightly different trees → combine predictions

① Bagging (Bootstrap Aggregating)

Random subsets of data — Bootstrap samples (with replacement)
Sample 1 — Model 1 Sample 2 — Model 2

For regression take avg. model 1 prediction + model 2 pred

for classification = mode(A, B, A) = A majority voting

→ Reduces variances — avg. noisy predictions

Random Subspace

- Fit N diffn decision trees by constraining each one to operate on a random subset of features.

Out of Bag Evaluation

- No need for validation separate or cross-validation
- Unbiased error estimate
- Ex: 100 data points & build RF with 10 trees.
 - For each tree, 63 data points IN-BAG, 37 out-of-BAG
 - For each data point, collect preds from all trees where it was OOB
 - majority vote or avg
 - OOB error proportion of times OOB pred $\neq$ true label
- Feature's importance = how much tree nodes that use that feature reduce impurity on avg.

③ Boosting

- All samples have same wt → reweight based on model's mistakes
- Ada Boosting Classifier: Update wts → $w_{i, \text{new}} = w_{i, \text{old}} \times e^{-\alpha_1 \cdot y_i \cdot h_1(x_i)}$

$$\alpha_1 = \frac{1}{2} \ln \left(\frac{1 - \epsilon_1}{\epsilon_1} \right) \Rightarrow \epsilon_1 = \text{Total error} = \text{sum of wts. of misclassified instances.}$$

Ex: Feature	Label	Initial wt	Classify Rule
1	+1	0.25 (+1)	For ex: +1 if $x \leq 1.5$ else -1
2	+1	0.25 (-1)	
3	-1	0.25 -1	
4	-1	0.25 +1	

Prediction +1

- ③ Compute error = $\epsilon_1 = 0.25$ — compute classifier wts ④

$$\alpha_1 = \frac{1}{2} \ln \left(\frac{1 - 0.25}{0.25} \right) = \frac{1}{2} \ln(3) \approx 0.549$$

- ⑤ update wts. for each instance i .
 - correctly classified $y_i \cdot h_1(x_i) = +1 = w_{i, \text{new}} = w_{i, \text{old}} \times e^{-0.549}$
 - misclassified $y_i \cdot h_1(x_i) = -1 = w_{i, \text{old}} \times e^{0.549}$

RF in regression Trees

- Split region — predict avg target value in each region
- Handle outliers & missing values robustly.
- may overfit if not regularized.
- feature space → region → predict mean of the region based on training data
- But split → split data left / right subsets → MSE reduces! choose the split

Example $\Rightarrow$ Var Red $\Rightarrow$ Var (parent) - $\leq$ Var (child)

~~Var red layer~~ = 474

Formulas

$$\text{Accuracy} = \frac{TP + TN}{\text{Total}}$$

Example

Actual \ Pred	Pred +ve	Pred -ve	Total
True +ve	TP	FN	Row total
True -ve	FP	TN	Row total

TN $\rightarrow$ Predicted 0, Actual 0

TP $\rightarrow$ Predicted 1, Actual 1

FP $\rightarrow$ Predicted 1, True 0

PN $\rightarrow$ Predicted 0, True 1

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (\text{Sensitivity})$$

$$F1 \text{ score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

$\Rightarrow$ Decision trees split data to maximize homogeneity in subsets - minimize impurity OR maximize information gain.

$$\Rightarrow \text{Gini impurity} = 1 - \sum_{i=1}^k p_i^2 \quad \rightarrow \text{If different class compute weightage}$$

$\Rightarrow$ compute Gini Gain wrt. to feature. Gini for each class used split to make a split.

$$\text{Gini Gain color} = \text{Gini at root node} - \text{Gini after split}$$

$\Rightarrow$ Consider feature with smaller Gini impurity

$$\text{Entropy Calculation} = - \sum_{i=1}^k p_i \log_2(p_i)$$

* Linear Equation Normal equation derive

LINEAR REGRESSION

$$\text{Hypothesis} = y = X\theta$$

matrix of i/p for
vector of params

$$\text{cost function} = J(\theta) = \frac{1}{2m} \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

vector dot product - with itself

$$\text{sub} = \frac{1}{2m} (X\theta - y)^T (X\theta - y)$$

$$\frac{dJ(\theta)}{d\theta} = \frac{1}{2m} X^T (X\theta - y)$$

$$\Rightarrow \frac{1}{m} X^T (X\theta - y)$$

$\Rightarrow$ minimize $\rightarrow$ set gradient to zero

$$X^T (X\theta - y) = 0 \quad \rightarrow \quad X^T X \theta = X^T y$$

$$\theta = (X^T X)^{-1} X^T y$$

$$A^{-1} = \frac{1}{ad-bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

/ determinant

if $ad-bc=0$
no inverse

$\Rightarrow$ Lasso $\rightarrow$ $\alpha \|\beta\|_1 = \text{MSE}(\beta) + \lambda \sum_{i=1}^n |\beta_i|$ L1 norm
 $\Rightarrow$ Ridge = L_2 penalty (Good Default)
 $\Rightarrow$ Elastic Net $\rightarrow$ Middle ground Weighted sum of both Ridge & Lasso regression $\rightarrow$ mix ratio λ
 $J(\beta) = \text{MSE}(\beta) + r \left(2\alpha \sum_{i=1}^n |\beta_i| \right) + (1-r) \left(\frac{\alpha}{m} \sum_{i=1}^n \beta_i^2 \right)$
 Few features $\rightarrow$ α

Bias - Variance Tradeoff

$\rightarrow$ Model too simple $\rightarrow$ Biased - not learned enough (some old / superstitions people!) $\rightarrow$ underfit. (High Bias)
 $\rightarrow$ Model too complex $\rightarrow$ Too rich people $\rightarrow$ minor changes $\rightarrow$ get inconsistent solution changes a lot (High Variance) $\rightarrow$ overfit

$\text{MSE} = \text{Bias}^2 + \text{Variance} + \text{Noise}$

- ① MSE $\rightarrow$ Avg. of (predict - actual)² $\Rightarrow E[(\hat{y} - y)^2]$
 - ② $y = f(x) + \epsilon \rightarrow$ noise with 0 mean var σ^2
 $\text{MSE} = E[(\hat{y} - f(x) - \text{noise})^2] = E[(\hat{y} - f(x) - \epsilon)^2]$
 - ③ $A = \hat{y} - f(x) = \hat{f}(x) - f(x) \quad \& \quad B = \epsilon$
 - ④ $\text{MSE}(x) = E[A^2] = E[A^2 - 2AB + B^2]$
 - ⑤ Apply linearity of expectation $= E[A^2] - 2E[AB] + E[B^2]$
- $\rightarrow E[A^2] = E[(\hat{f}(x) - f(x))^2] = (E[\hat{f}(x)] - f(x))^2 + \text{Var}[\hat{f}(x)]$
 $\downarrow$
Bias² + Variance
- $E[B^2] = E[\epsilon^2] = \text{Var}(\epsilon) = \sigma^2$

$\Rightarrow$ Locally weighted Reg (LWR) $\rightarrow$ non parametric Reg = Linger.
 Linear Regress + weighting function $\rightarrow$ wts to diff data points.

$\Rightarrow$ Logistic Reg $\rightarrow$ minimize log loss
 $J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(\hat{p}^{(i)}) + (1-y^{(i)}) \log(1-\hat{p}^{(i)})]$
 convex equation with global minima

$\Rightarrow$ In SVM $\rightarrow$ margins is the total distance b/w 2 support vectors on either side of the separating hyperplane

Margin = 2x Distance from hyperplane to support vector

⇒ Silhouette coefficient → evaluate quality of clusters — how well each point fits within its cluster compared to other

Example for point i .

$a(i)$ = avg distance point i → others same cluster

$b(i)$ = (avg distance point i → points in nearest (different) cluster)

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

⇒ 1 = well clustered

≈ 0 = on the boundary b/w clusters

-1 = probably in wrong cluster

variance → spread of data points → variance → spread of data points
 covariance → relationship between two variables → covariance → relationship between two variables
 correlation → strength of relationship between two variables → correlation → strength of relationship between two variables

$$E = \text{Bias} + \text{Variance} + \text{Noise}$$

$$E(y - \hat{y})^2 = E[(y - \hat{y})^2] = E[(y - \hat{y})^2]$$

$$E(y - \hat{y})^2 = E[(y - \hat{y})^2] = E[(y - \hat{y})^2]$$

$$E(y - \hat{y})^2 = E[(y - \hat{y})^2] = E[(y - \hat{y})^2]$$

$$E(y - \hat{y})^2 = E[(y - \hat{y})^2] = E[(y - \hat{y})^2]$$

$$E(y - \hat{y})^2 = E[(y - \hat{y})^2] = E[(y - \hat{y})^2]$$

$$E(y - \hat{y})^2 = E[(y - \hat{y})^2] = E[(y - \hat{y})^2]$$

$$E(y - \hat{y})^2 = E[(y - \hat{y})^2] = E[(y - \hat{y})^2]$$

$$E(y - \hat{y})^2 = E[(y - \hat{y})^2] = E[(y - \hat{y})^2]$$

variance → spread of data points → variance → spread of data points
 covariance → relationship between two variables → covariance → relationship between two variables
 correlation → strength of relationship between two variables → correlation → strength of relationship between two variables

$$E(y - \hat{y})^2 = E[(y - \hat{y})^2] = E[(y - \hat{y})^2]$$

$$E(y - \hat{y})^2 = E[(y - \hat{y})^2] = E[(y - \hat{y})^2]$$

variance → spread of data points → variance → spread of data points
 covariance → relationship between two variables → covariance → relationship between two variables
 correlation → strength of relationship between two variables → correlation → strength of relationship between two variables